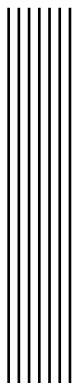


В.А. Лапшин

Инженерные онтологии

Современный подход

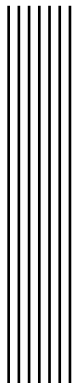
Москва • 2009



Оглавление

Оглавление	2
Предисловие	4
1 Что такое онтология	7
1.1 О чем эта книга	7
1.2 Определение онтологии	14
1.3 Типы онтологий	19
1.4 Что изложено в книге	30
1.5 Список литературы	34
2 Различные аспекты применения онтологий	37
2.1 Метафизика онтологии	37
2.2 Онтологии верхнего уровня	49
2.3 Методология построения онтологий	64
2.4 Список литературы	82
3 OWL— язык описания онтологий для Web	88
3.1 Необходимость в OWL	88
3.2 Язык XML	90
3.3 Язык RDF	95
3.4 Идеология OWL	114
3.5 OWL редактор Protege	124
3.6 Реализации хранилищ онтологий на основе OWL и RDF .	135

3.7	Список литературы	137
4	Ontolingua— Web-сервер библиотек онтологий	140
4.1	Концепция «разделяемых знаний»	140
4.2	Описание языка KIF	142
4.3	Ontolingua KIF	163
4.4	Онтология библиографических данных	176
4.5	Онтологический Фреймовый Редактор	181
4.6	Список литературы	191
5	Сус— библиотеки онтологий для AI	193
5.1	Что такое Сус	193
5.2	Язык СусL	196
5.3	Система логического вывода	216
5.4	Онтология системы Сус	226
5.5	Проект OpenCус	236
5.6	Список литературы	241



Предисловие

Развитие информационных технологий последней четверти прошлого века характеризовалось разработкой технологий баз данных и повсеместного их внедрения. В начале этого века, подобные процессы наблюдаются в разработке технологий, связанных с онтологическим моделированием. У большинства исследователей уже сформировалось понимание того, что использование библиотек онтологий в организации информационных процессов в ближайшие десятилетия будет таким же обычным делом, как использование баз данных сейчас. Между тем, литература на русском языке об информационных инженерных онтологиях и технологиях их использования практически отсутствует. Данная книга предназначена восполнить этот пробел.

Книга представляет собой обзор понятий, языков и методов, связанных с применением онтологий в информатике. В книге представлены наиболее крупные мировые системы и проекты использующие информационные технологии, основанные на онтологиях.

Целью автора книги было написание ясного введения в проблематику использования онтологий в информационных системах, в Веб, в задачах представления и использования знаний, для людей, интересующихся информатикой и впервые знакомящихся с этой темой.

К сожалению, рассказать о технологиях онтологического моделирования так, чтобы это было понятно неподготовленному читателю, довольно трудно. Дело в том, что для описания знаний¹ используется

¹Онтологии, упрощенно говоря, представляют собой описания знаний.

много методов, которые требуют довольно объемного предварительного изучения. Например, наиболее известные текущие реализации систем построения онтологий основаны на языках исчислений предикатов первого и второго порядков. Ясно, чтобы понять, как устроена та или иная система построения онтологий, необходимо сначала иметь понимание того, что лежит в основе данной системы.

Другое препятствие для быстрого понимания более фундаментально. Состоит оно в том, что построение онтологии знаний представляет собой ни что иное, как процесс моделирования этих знаний, т.е. построения модели мира решаемой задачи. Построение модели—непростой и требующий определенных навыков процесс. Хорошая модель должна, с одной стороны, быть достаточно общей, чтобы должным образом отражать представление о мире задачи, а с другой стороны, содержать все необходимые для решения задачи детали. У неподготовленного читателя неизбежно будут возникать вопросы относительно того, почему модель была построена именно таким образом, зачем одни понятия были описаны в модели, а другие, кажущиеся важными, наоборот, были исключены из рассмотрения и т.д. Чтобы должным образом понимать мотивы, стоящие за тем или иным решением, необходимо некоторое погружение в предмет.

Процесс построения модели называется *концептуализацией*. Сам этот термин говорит о том, что в процессе построения модели выделяются концепты—понятия, лежащие в основе модели, и взаимоотношения между этими понятиями. Читатель также должен иметь некоторое понимание того, что такое понятие и как оно выражается формальным образом. Для прояснения этих вопросов приходится обращаться к философии. Таким образом, в книге также необходимо дать некоторое представление о философских идеях, лежащих в основе процесса концептуализации.

Наконец, не следует забывать, что книга должна дать представление о наиболее известных реализациях систем построения онтологий, т.е. некоторым образом, представлять собой обзор этих систем. Можно описать каждую такую систему в нескольких абзацах, но к сожалению, такой небольшой объем информации не даст читателю ничего, кроме понимания того, что такая система существует. Чтобы дать читателю более полное представление о системе, нужна целая глава. Таким образом, необходимо погружение в систему.

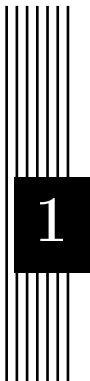
Книга представляет собой введение в идеи, связанные с понятием онтологии, а также одновременно является обзором наиболее известных реализаций систем построения онтологий. Текст книги можно читать без всякой подготовки. Первая глава представляет собой элементарное

изложение основных идей, показывающих необходимость онтологий в современной инженерной науке. Также, в первой главе приведены примеры различных типов онтологий. Начинать чтение книги надо обязательно с этой главы. Чтение остальных глав можно производить в любом порядке. Книгу также можно использовать как справочник по системам построения онтологий.

Работа над данной книгой велась при частичной поддержке фонда РФФИ, грант **09-07-00079-а**.

Автор выражает благодарность Евгению Михайловичу Бениаминову за помощь и ценные замечания, высказанные в ходе работы над книгой. Без этой помощи книга вряд ли была написана.

Москва, сентябрь 2009 года.



1 Что такое онтология

1.1 О чем эта книга

Эта книга представляет собой попытку систематически изложить различные методы применения понятия «онтология» в инженерной науке. Онтология сейчас модный термин, который используется (к месту или нет) во многих областях человеческой деятельности. Слово «онтология» можно встретить не только в научных трудах, но и в газетных статьях или телевизионных передачах. В общем, слово, так сказать, «на слуху». Но, как и всякое новое понятие, термин «онтология» еще не получил ясной и четкой интерпретации¹. Данный труд является попыткой заполнить этот пробел, хотя и не претендует на то, чтобы изложенные в нем подходы рассматривались, как истины в последней инстанции. В книге излагается видение, которое сложилось в процессе исследований в научном коллективе, в котором автор является действующим членом.

В инженерии, «онтологические» методы обычно применяются для построения моделей процессов. Инженерная модель процесса и есть онтология. Точнее, онтология представляет собой *описание* этой модели.

¹Среди специалистов до сих пор ведутся дискуссии по поводу терминологии, связанной с понятием «онтология». В России, в мае 2008 года в г. Звенигород был проведен симпозиум под названием «Онтологическое моделирование», посвященный анализу существующих направлений в данной области. Одной из целей этого симпозиума была попытка достичь договоренности о том, что можно называть онтологией, а что нет. Труды симпозиума изданы в [23].

Такие описания моделей обычно *формальны*, т.е. сделаны на разработанном специально для этих целей языке, конструкции которого всегда интерпретируются точно и однозначно. Это позволяет сохранять описание в компьютерной памяти с тем, чтобы в дальнейшем использовать его в другой задаче или проверить с помощью компьютера, не содержит ли описание логических противоречий.

На самом деле, задача автоматизированного обмена формальными описаниями моделей явилась главным побуждающим фактором начала исследований в области применения онтологий. Рассмотрим области инженерии, где такие задачи возникают.

Впервые, задача построения формального описания модели возникла в программных системах управления базами знаний. Базы знаний— это систематизированное собрание фактов, которое управляется разработанной специально для этих целей программой. База реализуется как *экспертная система*, т.е. программа, которая выступает в качестве специалиста-эксперта в некоторой тематике. Например, сейчас очень популярны экспертные системы в медицине. Такая база знаний содержит такое большое количество медицинских фактов, что один человек не в состоянии запомнить. Экспертной системе могут быть поданы на вход результаты анализов крови и программа, сопоставив факты своей базы знаний, может выдать по результатам этих анализов диагноз больного. Экспертную систему можно дообучать, добавляя в нее новые факты. Теперь, предположим, что существуют две такие экспертные системы, сделанные различными производителями и расположенные в разных медицинских учреждениях, и министерство здравоохранения решило создать новую экспертную систему на основе двух существующих. Если бы эксперты были людьми, а не машинами, то все было бы просто: они бы встретились и рассказали друг другу все, что знают. С машинами ситуация иная— они друг другу просто так рассказать ничего не могут. Поэтому, необходимо формально описать содержимое базы знаний каждой экспертной системы с тем, чтобы это описание было понятно программе, которая реализует их объединение. Здесь возникают следующие затруднения:

1. Надо разработать язык, который был бы понятен каждой из трех участвующих программ. Этот язык должен быть достаточно формальным для того, чтобы описания баз знаний (онтологии) однозначно интерпретировались машинами.
2. Построить описание содержимого базы знаний на этом языке. Понятно, что человек такое описание сделать не в состоянии, так как

он не в состоянии держать в уме содержимое всей базы знаний экспертной системы. Значит, описание должна делать программа.

3. По описанию базы знаний, сделанному на разработанном языке, добавить новые факты в базу знаний третьей экспертной системы. Это также должна сделать программа.

В полном соответствии с традициями инженерной науки, возникла мысль стандартизировать язык описания знаний, чтобы больше не тратить усилий на его разработку. Тогда, второй и третий пункты из списка выше свелись бы просто к написанию переводчиков с внутреннего языка базы знаний в этот язык, и с языка описания на язык представления знаний в экспертной системе. Это и было сделано (язык обмена знаниями называется KIF— Knowledge Interchange Format [19]), но про то, каким образом это было сделано, мы поговорим ниже в четвертой главе.

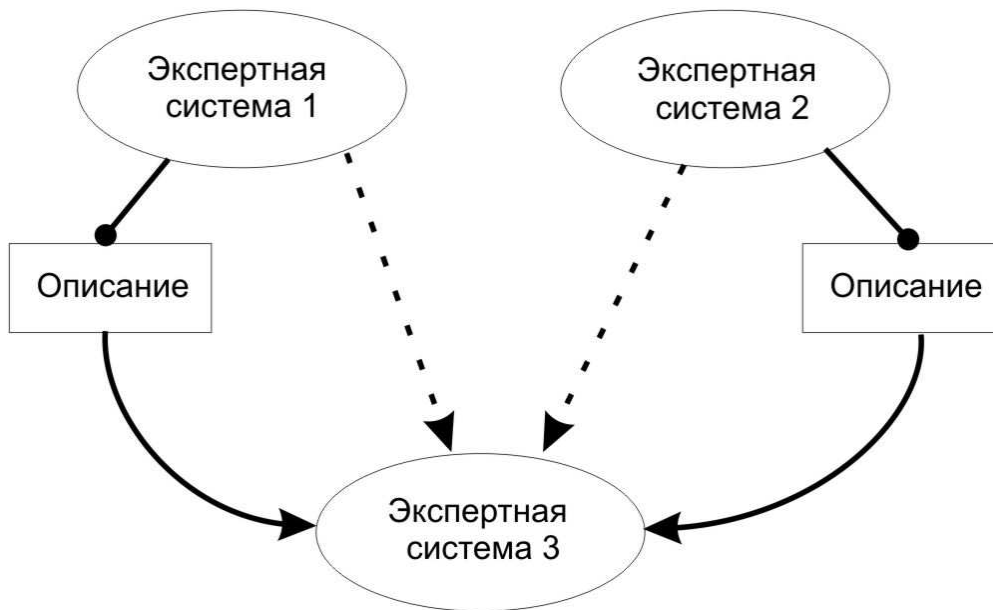


Рис. 1.1: Обмен знаниями между экспертными системами.

Потребность построения описаний моделей возникла также в задаче поиска Web-страниц по запросам пользователей. В самом начале появления Web, поисковые программы просто искали в Web страницы, содержащие текст запроса. Например, если пользователь вводил запрос «Владимир», то поисковая программа искала в текстах страниц слово «владимир» и показывала страницы, содержащие это слово

в достаточных количествах. Но, слово «владимир» имеет неоднозначное истолкование. Например, это может быть имя известного человека: «Владимир Путин» или «Владимир Ленин», в этом случае, необходимо показать страницы об этих людях. Также, это может быть название города Владимир, тогда надо показать страницы об этом городе. Достаточно умная поисковая программа могла бы иметь достаточно большую базу фактов, чтобы «знать» о различных значениях слова «владимир». Но, это бы ничего такой программе не дало до тех пор, пока создатели сайтов не предоставили описания содержания своих страниц поисковым программам. Предположим, что сайт о городе Владимир имеет такое описание. Тогда, поисковая программа может уточнить у пользователя, не имеет ли он ввиду в запросе «город Владимир» и, в случае согласия, вывести на первом месте в списке найденных страниц страницу этого сайта вместе с ее описанием. Описания содержимого Web-страниц представляют собой *онтологии* этих страниц. Web, в котором страницы имеют описания их содержимого, называет Умный Web (Semantic Web). Для построения онтологий необходим специальный язык, который был бы понятен поисковым программам. Такой язык создан и называется OWL (Web Ontology Language [10]). Мы подробно обсудим этот язык и проблематику построения описаний Web-страниц в третьей главе.

Построение описания знаний необходимо производить в любой объемной системе управления знаниями. Если разработчики базы знаний планируют ее пополнение новыми фактами, то для этого важно сначала систематизировать содержание этой базы знаний. Надо выделить в базе знаний разделы, описать их содержание, а также способы связей различные разделов между собой. В этом случае, пользователи будут добавлять в систему новые знания в нужном месте. Например, если система содержит знания по медицине и по биохимическим процессам, то во время введения информации о новом лекарстве, необходимо будет как-то связать знания о том, при каких симптомах применять это лекарство, со знаниями о том, какие химические вещества содержатся в этом лекарстве и какое физиологическое действие производит прием данного лекарства. Для этого, необходимо построить онтологию базы знаний (т.е. ее описание) и в этой онтологии связать различные разделы между собой. Тогда, пользователь просто заполнит нужные поля при вводе информации, и добавленные знания будут связаны в стройную и логичную схему. Здесь, мы видим, что описание знания играет важную роль в его *систематизации*. В главе 5, на примере базы знаний под названием Сус [13], мы рассмотрим, как систематизируются объемные базы знаний.

Систематизация знаний— задача совсем не простая. Кроме тех зна-

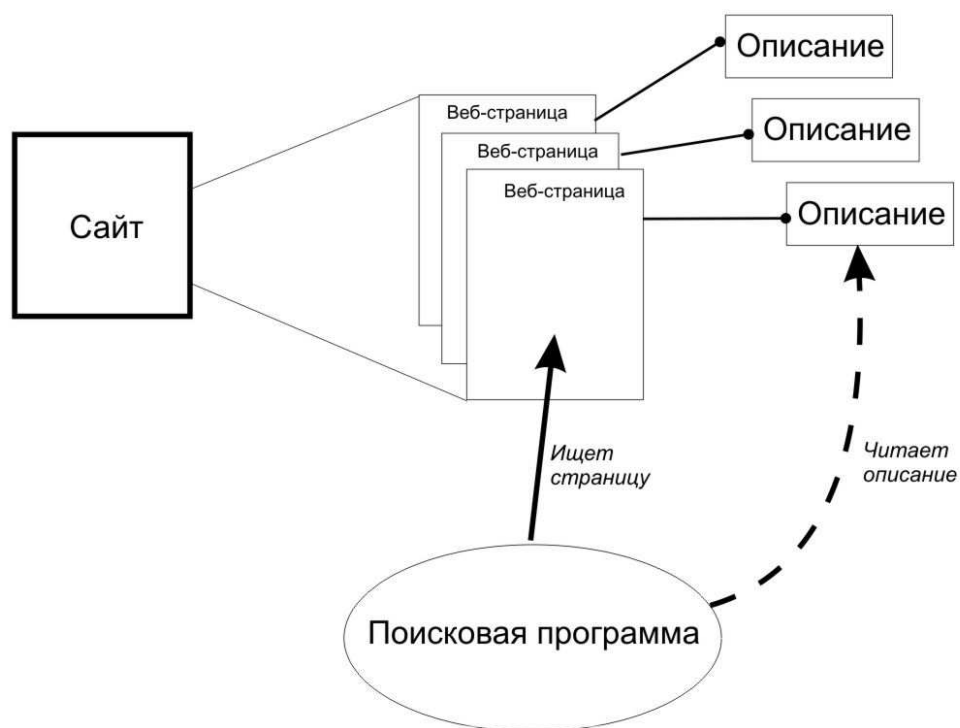


Рис. 1.2: Описания Web-страниц для поисковых программ.

ний, которые непосредственно содержаться в базе, необходимы еще так называемые *общие знания*. Человек о таких знаниях даже не задумывается, они подразумеваются, как само собой разумеющееся. Например, человек знает, что люди ходят вверх головой и ногами по земле и, вообще, то что они ходят есть следствие существования силы притяжения. Человеку не придет в голову, что можно пройти сквозь стену, потому что он имеет знание о том, что стена— это непроницаемый объект. Для базы, содержащей только конкретные знания, все эти очевидные для человека понятия недоступны. Поэтому, родилась мысль о том, чтобы формально описать основные свойства мира,— так называемые общие знания, и применять эти знания в различных базах конкретных знаний. Эти описания обычно называют *онтологиями верхнего уровня* (top-level ontologies). Типичная база знаний чаще всего содержит три уровня знаний:

1. Общие или абстрактные знания, описанные в онтологии верхнего уровня.

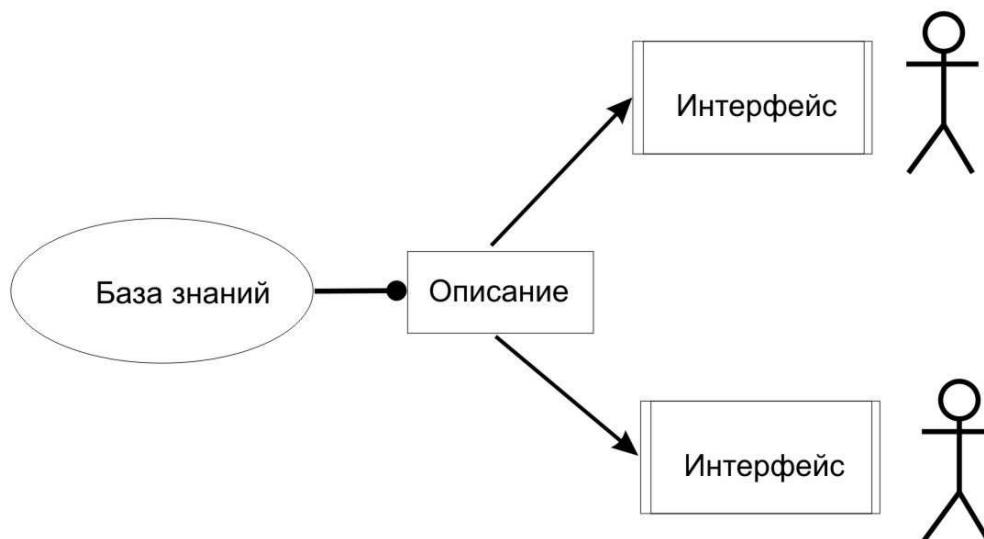


Рис. 1.3: *Онтология как интерфейс к базе знаний.*

2. Знания о конкретной предметной области (domain specific knowledge). Например, знания по географии, политологии или медицине.
3. Конкретные знания, добавляемые в базу знаний пользователями или программными агентами.

Разделение знаний на уровни вообще свойственно человеку, об этом мы подробно поговорим ниже в начале второй главы (см. раздел 2.1). Существуют различные подходы к описанию общих знаний, эти подходы мы рассмотрим подробнее в разделе 2.2 второй главы.

Сейчас, онтологии применяются во многих областях инженерии. Выше, мы рассмотрели несколько примеров, где без описания знаний сделать вообще ничего нельзя. Но, существуют также сферы, где важность формального описания знаний для машин еще не до конца осознана и, потому, применение онтологических подходов в этих областях выглядит неожиданно. Рассмотрим, например, такую область, как лингвистика. Знания о языке в наш век описаны достаточно формально во многих учебниках и с успехом используются различными специалистами. Но, оказывается, и в этой области стали необходимы описания знаний в удобной для машин форме. Так, известно, что создание ком-

пьютерного переводчика с одного языка на другой,— задача, имеющая большую важность. Решение этой задачи сэкономило бы человечеству много усилий. Ежегодно, в мире выпускается большое число специалистов по переводу, государства тратят на обучение таких специалистов достаточно объемные средства. Если бы был создан машинный переводчик, то эти средства можно было бы направить на подготовку специалистов в других отраслях. Для решения задачи автоматического перевода, стала очевидной необходимость формального описания знаний о языках. Разработчики предыдущих поколений программ машинного перевода пытались создать эти программы как обычные приложения. Создавалась программная модель, логика перевода описывалась программным кодом, а словарь обычно сохранялся в базе данных. Но, оказалось, что знания о языке настолько объемны, что закодировать их непосредственно в программной модели весьма затруднительно. Кроме того, затруднения имеются в самой формализации знаний о языке— многие трудноуловимые аспекты этих знаний до сих пор не описаны формально. В связи с указанными затруднениями, возникла мысль создать описание знаний о языке в машинном формате с тем, чтобы затем использовать это описание в различных программах, в том числе, и в программах машинного перевода. Такие проекты существуют, например есть проект описания знаний об английском языке под названием WordNet [17].

Вообще, как представляется автору, происходящий сейчас процесс внедрения формального описания знаний в различные области инженерии подобен тому, который происходил в 70-80 годах прошлого века с базами данных. В 60-х годах прошлого века была осознана необходимость отделения описания данных от самих данных. Описание данных— это так называемая схема базы данных (концептуальная схема). Была создана формальная теория таких описаний [см., например, 28] и, затем, были созданы приложения, которые могли работать с такими описаниями— так называемые серверы баз данных. Подобные процессы, только относительно описания знаний, наблюдаются и в наше время. Подобно тому, как введение баз данных позволило заменить машинами человека (и, тем самым, существенно увеличить объемы обрабатываемой информации) в различных областях, связанных с обработкой данных, введение онтологий позволит машинам выполнять работу человека в некоторых областях, связанных с более интеллектуальной деятельностью.

1.2 Определение онтологии

Термин «онтология» не нов и используется в философии уже около 600 лет. Термин был предложен средневековым философом из города Марбурга Рудольфом Гоклениусом. В 1613 году Гоклениус выпустил знаменитый «Философский словарь» [2], и там, наряду с термином «антиномия», предлагался термин «онтология». В 1656 году, этот термин (в виде эквивалентного термина «онтософия») был применен немецким философом Иоган Клаубергом в его работе [4] в качестве эквивалента понятию «метафизика». Позже, немецкий философ Христиан фон Вольф отделил семантику терминов «онтология» и «метафизика» друг от друга. Сейчас термин «метафизика» используется для обозначения раздела философии, изучающего первоначальную природу реальности, или даже вообще для обозначения всей реальности, выходящей за рамки физического.² Онтология же стала означать более узкую область метафизики, посвященную смыслам предметов (понятиям) их взаимоотношениям друг с другом.

Слово «онтология» образовано от двух корней древнегреческого языка: *οντος*— то, что существует (сущее) и *λογος*— учение (наука). В современной философии термин понимается двояко. С одной стороны, этот термин означает науку о бытии. С другой стороны, термином «онтология» обозначают некоторые стороны бытия, которые, как считается, лежат в основании последнего. Приведем основное определение:

Определение 1.1 *Онтология (в философии)— это раздел философии, изучающий проблемы бытия.*

Читатель, вероятно, уже убедился, что понимание термина «онтология» в инженерной науке отличается от философского. В инженерии, понятие «онтология» было сформировано следующим образом. Сначала, была осознана необходимость отделения знаний, которые задаются моделью программной системы, от поведения этой программной системы. Затем, на основе этого понимания, сформировалась идея о необхо-

²Метафизикой в школе Аристотеля называли те стороны бытия, которые не описывались в аристотелевской «Физике»— труде, посвященном описанию физического мира и его законов. Существует также и иная точка зрения на происхождение этого термина. Рассказывают, что в Королевской Библиотеке Лондона книги по «Физике» Аристотеля стояли на передней полке, а другие его произведения— позади. Древнегреческий корень «мета» означает «за», отсюда, как говорят, и возникло название тех трудов Аристотеля, что находились на задней полке. В этом смысле термин «метафизика» означает «за физикой» в буквальном смысле.

димости строить формальные описания знаний программных систем. Рассмотрим подробнее, как это происходило.

Осознание необходимости отделения описания знаний от самих знаний начало происходить в конце 80-х годов прошлого века. Вместе с осознанием этой необходимости начался пересмотр самого понятия «знание», применительно к компьютерным интеллектуальным системам.

До этого [36], знание рассматривалось с функциональной точки зрения:

Знание— это нечто, что может быть объяснено (программному) агенту, чтобы его поведение могло быть вычислено в соответствии с принципами рационального поведения.

Эта точка зрения выглядит вполне логичной. Если для человека знание— это прежде всего вера в истинность или, если угодно, в полноту представлений этого человека о мире (или о какой-то его части), то для машины понятие веры совершенно не применимо. Поэтому, знание рассматривается как информация, заставляющая программную систему действовать рационально и предсказуемо. Подразумевается, что эта информация приходит от некоего эксперта,— человека, обладающего знанием. Позже [27], было предложено рассматривать знание, как модель представлений о мире, а не модель состояния ума некоего эксперта. Согласно этому подходу, знание компьютерной системы— это модель объективной реальности, точнее, некоторой ее части, называемой *предметной областью*. Но, тогда можно отделить модель описания мира от поведения программного агента.

Термин «онтология» впервые появился в работе Томаса Грубера [30]. Грубер решал инженерную задачу создания механизма взаимодействия программных систем баз знаний друг с другом. Для решения этой проблемы были предложены следующие методы:

- Выделить в системе управления знаниями уровень так называемого *декларативного знания*. Это знание о том, из чего состоит мир, реализованный в данной системе, т.е. декларативное знание представляет собой описание данного мира. В противоположность декларативному знанию можно ввести *процедурное знание*, например, знание о том, каким образом на основании описания мира делать логические выводы относительно тех или иных его свойств. Иначе говоря, предлагалось отделить знание от его обработки.
- База знаний предоставляет описание своего декларативного знания другим программам управления знаниями. Причем, такое

описание должно быть понятно, как человеку, так и машине, и потому, выдается в двух видах:

1. Канонической форме, представляющей собой описание знания на обычном языке логики предикатов.
 2. В форме *онтологии*, как ее представлял в тот момент Грубер, т.е. в виде набора описаний *термов* (классов, отношений, констант и т.д.) и *определений*, связывающих эти термы друг с другом.
- Построить, на основе выделенных описаний, *библиотеки онтологий*, которые можно было бы использоваться в различных базах знаний.

Таким образом, Т. Грубер является автором термина «онтология» в инженерии.

Задача построения описания знания является довольно специфической. Грубер даже выделил для этой задачи отдельный термин— «спецификация концептуализации». Здесь под «концептуализацией» понимается «абстрактный, упрощенный взгляд на мир, который используется людьми для осуществления некоторой цели» [31]. Особенность задачи концептуализации заключается в том, что для обмена знаниями между программными системами необходимо явно специфицировать их концептуализацию, т.е. построить описание этих знаний, причем в достаточной степени формальное, чтобы его понимали другие системы. Результат такой спецификации был назван Грубером термином «онтология».

Таким образом, понятие *инженерная онтология* можно определить как спецификацию (формальное описание) некой концептуализации (представления предметной области исследуемой задачи так, как это необходимо для данной задачи). Если со спецификацией здесь все более-менее ясно, то что такое *концептуализация*³ не совсем понятно. Грубер считал, что концептуализация проводится в терминах классов и атрибутов. Мир исследуемой задачи представляется в виде понятий, которые описываются классами, вместе с их свойствами (атрибутами), и конкретными объектами,— экземплярами классов. Например, проектируя программную систему, содержащую информацию об автомобилях, естественно ввести классы, представляющие марки автомобилей (Тойота, Лада, Ниссан и т.д.), а конкретные автомобили рассматривать, как

³ Следует отметить, что английский термин *concept* означает ни что иное, как *понятие*. Отсюда можно вывести значение термина *концептуализация*, как построение системы понятий, отражающей данную предметную область

экземпляры соответствующих классов. Такой подход популярен и на нем основаны принципы объектно-ориентированного программирования.

Автор полагает, что сведение определения концептуализации к представлению только в виде классов и их атрибутов, несколько узко. В разделе 2.1 приводится пример определения понятий посредством так называемых *прототипов*— типичных представителей понятия. Так, для понятия «инструмент» имеется прототип «молоток», а для понятия «плод» прототипом является «яблоко». Разумеется, у некоторых людей могут иметься другие прототипы. Например, для понятия «плод» часто в качестве прототипа называют «апельсин». Это не устанавливает принципиальных ограничений на использование прототипа, как представителя понятия. Таким образом, понятия можно описывать с помощью их типичных представителей— прототипов, и определять, принадлежит или нет конкретный объект данному понятию с помощью меры *семантического расстояния* данного объекта от прототипа понятия. Существуют понятия, которые только так и можно ввести. Вопросы, связанные с различными методами определения понятий подробно обсуждаются в разделе 2.1.

Концептуализация— это всегда упрощенный взгляд на мир. Из предметной области выделяются только те элементы, которые необходимы для данной задачи⁴, другие свойства предметной области не рассматриваются. Приведем хрестоматийный пример. Пусть, в качестве предметной области выступает конкретный дом. Для постройки дома задействуют несколько групп специалистов, среди которые имеются архитекторы, поставщики материалов и строители. Архитекторы рассматривают дом, как единое целое, акцентируя внимание, главным образом, на том, как различные части (комнаты, этажи) составляют это целое. Строители также рассматривают дом, как единое целое, но внимание акцентируется, в основном, на производственных процессах: сколько человек и какое оборудование отрядить на постройку этажа, сколько времени необходимо для постройки и т.д. Поставщики материалов вообще не рассматривают дом, как единое целое. В представлении постав-

⁴Мы будем в таких случаях говорить: *цели*, а не задачи. Ведь, понятно, что за каждой задачей стоит некая цель, ради достижения которой эта задача поставлена. Человеческий ум устроен таким образом, что он постигает мир только через упрощенное представление, которое обусловлено некоторой целью. Это представление называется *абстракцией*. Например, когда рассматривается конкретный автомобиль, то совсем неважно, скажем, каков этот автомобиль на вкус. Хотя, это свойство (вкус) определенно присутствует у всех автомобилей, оно в данном случае (т.е. применительно к понятию «автомобиль») не важно.

щика, дом— это вместилище нескольких тон цемента, бетона, стали и других строительных материалов. Таким образом, цель определяет то, как будет проводиться концептуализация.



Рис. 1.4: Различные концептуализации объекта «дом».

Термин «концептуализация» использовался Грубером также и для обозначения процесса построения онтологии. Результат концептуализации мы будем называть *концептуальной схемой*. Этот термин используется в базах данных примерно в том же значении, но в частном случае определения схемы базы данных для некоторой производственной задачи. Имеется, также, некоторое различие в терминологии. Выше, мы определили онтологию, как спецификацию концептуализации предметной области задачи. Но, в философии, термином «онтология» обозначается представление *реальности* в виде набора понятий и связей между ними, т.е. по сути, концептуализация *реальности*. В инженерии производится концептуализация предметной области инженерной задачи, причем цель этой концептуализации, обычно, сугубо утилитарная и связана с содержанием решаемой инженерной задачи. Таким образом, имеется явное разделение целей:

- Онтология в философии создается для изучения свойств реальности.

- Онтология в инженерии создается для решения инженерной задачи.

Это расхождение в целях концептуализации явилось мотивацией для следующего предложения: термином *онтология* называть только спецификации концептуализаций реальности, сделанные специально с целью построения такой концептуализации, а все остальные спецификации концептуализаций обозначать термином *концептуальная модель* [5]. Соответственно, онтология— это описание реальности в виде формальным образом специфицированной схемы понятий, а концептуальная схема— то же, но только применительно к предметной области некоторой инженерной задачи. Автору такое разделение кажется излишним, но конечно, читатель вправе применять эту терминологию.

Для обозначения процесса построения концептуальной схемы используется также термин *моделирование*. Как уже было сказано выше, многие специалисты в области инженерной онтологии различают понятия *онтологическое моделирование* и *концептуальное моделирование* [29; 37; 34].

Приведем точные определения концептуальной схемы и инженерной онтологии.

Определение 1.2 *Концептуальной схемой инженерной задачи называется абстрактный упрощенный взгляд на предметную область данной задачи, выражающийся в выделении только тех свойств предметной области, которые полезны для целей данной задачи, и отбрасывании из рассмотрения остальных ее свойств.*

Определение 1.3 *Инженерная онтология— это формальное описание (спецификация) концептуальной схемы инженерной задачи.*

1.3 Типы онтологий

В данном разделе мы рассмотрим различные типы онтологий, начиная от простейших таксономий и заканчивая сложными онтологиями, поддерживающими изменения их содержания.

Классификации

Классификация представляет собой простую форму спецификации концептуальной схемы, в которой все элементы предметной области раз-

Природный мир
Царство
Класс
Отряд
Род
Вид
Разновидность

Таблица 1.1: *Элементы Линнеевской Иерархии*

бываются на типы⁵ и между введенными типами вводится отношение иерархии. Тип является надтипом другого, если класс, который задается данным типом, содержит все экземпляры класса подтипа. Из определения отношения иерархии через отношение подмножества следует, что отношение иерархии транзитивно, т.е. если тип A является подтипом типа B , а тип B — подтипом типа C , то тип A будет подтипом типа C . Классификация основана на аристотелевской традиции определения понятий как коллекций свойств.

Одной из наиболее известных систем классификации является так называемая *Линнеевская Иерархия* [6], названная так в честь шведского натуралиста Карла Линнея [1]. Линнеевская Иерархия представляет собой научную классификацию природного мира. Корневым типом является «природный мир», который изначально был разделен Линнеем на три подтипа, называемых *царствами*: «минеральное», «растительное» и «животное». Сейчас, в биологической классификации царствами являются восемь подтипов типа «природный мир», мы на них подробно останавливаться не будем. Непосредственным подтипом типа «царство» являлся тип «класс». Изначально, в Линнеевской Иерархии имелось всего шесть классов: млекопитающие, птицы, гады, рыбы, насекомые и черви. Подтипом класса являлся «отряд», его подтипом— «род», подтипом рода являлся «вид» и элемент предметной области «природный мир» назывался «разновидность». Уровень иерархии называется рангом, таким образом, в линнеевской классификации существует шесть рангов, перечисленных в порядке убывания ранга в таблице 1.1.

Например, элемент предметной области «природный мир» (разновидность) Петр Иванов будет классифицирован следующим образом:

⁵Типы имеют атрибуты. Для типа, значение некоторых атрибутов должно быть равно одной и той же величине (такие атрибуты называются существенными), а значения других атрибутов могут быть любыми. Например, можно ввести тип «прямоугольник», для которого такие атрибуты, как длина сторон, площадь и т.п. могут меняться. Но, атрибут «углы» всегда имеет значение 90 градусов для всех углов.

царство «животное», класс «млекопитающие», отряд «приматы», род «человек», вид «человек разумный». Сейчас биологическая классификация значительно изменилась, появились новые ранги и существенно расширилось количество признаков (свойств) каждого ранга, но принципы классификации остаются неизменными.

Для классификации с иерархией типов часто используется термин *таксономия*. Таксономия— это классификация, отношение иерархии в которой может быть представлено в виде дерева типов, т.е. в таксономии у любого типа не может быть более одного надтипа. У специалистов по инженерным онтологиям для классификаций часто используется как синоним новый термин «легкая онтология» (lightweight ontology). Этот термин очевидным образом подразумевает, что также существуют и «тяжелые онтологии». Последним термином обозначаются такие онтологии, в которых, кроме выделения иерархии типов между элементами, существуют более сложные связи, выражаемые, обычно, специальными *соотношениями* и *функциями*. Существует большой класс задач, для моделирования которых необходимо использовать тяжелые онтологии. В этом смысле, моделирование тяжелых онтологий не является чем-то из ряда вон выходящим. Простые примеры тяжелых онтологий будут приведены чуть ниже в разделе «Онтологии с соотношениями».

Интересно, что аналогии линнеевской классификации неожиданным образом проявились в методологии программирования. Одной из наиболее модных на сегодня парадигм программирования является так называемое *объектно-ориентированное программирование* (ООП). В ООП оперируют объектами, которые представляют собой экземпляры классов. Класс в ООП— это коллекция атрибутов и методов, имеющая уникальное имя. Атрибут класса имеет тип, а конкретный экземпляр класса заполняет свои атрибуты конкретными значениями. Метод класса— это некий алгоритм, который может быть вызван для данного экземпляра класса и получает набор конкретных параметров, основанных на переданных в метод значениях и значениях атрибутов данного экземпляра класса. Между классами можно установить отношение иерархии, используя так называемый механизм *наследования*. При объявлении типа класса указывается, что он «наследуется» от одного или более родительских типов, это означает, что все атрибуты и методы родительских типов являются теперь атрибутами и методами дочернего класса. Очевидно, что такая методология языка программирования задает описание концептуальной схемы программы в виде классификации, что упрощает процесс программирования, особенно для объемных программ, т.к. позволяет разделить элементы программы на модели с четко выраженным интерфейсом и хорошо специфицированным про-

токолом обмена данными между ними. Теория ООП была заложена в конце 70-х годов и первопроходцем в этом была Барбара Лисков [35]. Ее подход основывался на аналогии с линнеевской классификацией и из нее же была унаследована терминология.⁶

Другой известный пример классификации представляет собой *Периодическая система химических элементов Д.И. Менделеева* [8]. Корневым типом в этой классификации является «химический элемент». Химический элемент имеет несколько атрибутов, главным из которых является *атомный вес*. Остальные свойства химического элемента определяются его атомным весом. Периодическая система не является таксономией, так как ее классификация не иерархическая, но основана на важнейшем понятии «периода». Сущность открытия Менделеева заключалась в том, что с ростом атомного веса химических элементов их свойства меняются не монотонно, а периодически. После определенного количества разных по свойствам элементов, расположенных по возрастанию атомного веса, свойства начинают повторяться. Например, натрий похож на калий, неон похож на аргон, а золото похоже на серебро и медь. Разумеется, свойства не повторяются в точности, основные или, как говорят, *существенные* для данного периода, характеристики остаются неизменными. Таким образом, химические элементы можно классифицировать по принадлежности к периоду. Кроме того, порядковый номер химического элемента в периоде также важен: все элементы с одним и тем же порядковым номером в периоде обладают похожими свойствами. Так, неон и аргон представляют последние элементы своих периодов и принадлежат семейству инертных газов. Один и тот же химический элемент, например неон, принадлежит первому периоду, т.е. типу «первый период», но также и принадлежит и типу «инертные газы». Следовательно, классификация периодической системы не является таксономией. Это пример более сложного вида классификации, показывающий, что не все классификации представляют собой таксономии.

Сейчас в системах Web-поиска интенсивно используются классификации Web-сайтов, в которых описываются иерархически связанные между собой категории поиска. Обычно, такая классификация представляет собой расширенную таксономию, где присутствует дерево поисковых категорий, каждый элемент которого может иметь дополнительные свойства. Такая классификация, фактически, представляет со-

⁶Этим, вероятно, можно объяснить, что методология проектирования программных систем в 80-х и 90-х годах страдала, если можно так выразиться, излишним биологизмом.

бой таксономию Интернета, построенную с целью оптимизации поисковых задач. Например, классификация компании «Яндекс» называется «Яндекс Каталог» [12] и представляет собой дерево не более четырех уровней иерархии. Типы в этой классификации называются «рубриками». Существуют следующие рубрики верхнего уровня:

- Hi-Tech
- Работа
- Учеба
- Дом
- Общество
- Развлечения
- Отдых
- Культура
- Спорт
- СМИ
- Производство
- Бизнес
- Справки
- Авто
- Порталы

Подобные классификации существуют во всех популярных поисковых системах.

Онтологии с соотношениями

Хотя классификации представляют собой наиболее часто встречающийся вид онтологий, все же существуют задачи, концептуальные схемы которых не укладывается в рамки простой классификации.

Рассмотрим простой пример. Пусть проектируется онтология геометрических фигур и в этой онтологии необходимо описать тип «квадрат». Этот тип можно описать двумя способами:

1. Как прямоугольник, у которого стороны равны друг другу.
2. Как ромб с равными углами.

В обоих случаях не обойтись простой классификацией, необходимо как-то задать соотношения между сторонами прямоугольника или углами ромба.

Вообще, часто типы очень трудно описать простым перечислением атрибутов. Например, легковой автомобиль можно описать, как автомобиль, масса которого не превышает трех с половиной тонн. Иначе говоря, легковой автомобиль задается посредством ограничения на значение своего атрибута. Это наиболее часто используемый тип соотношений.

Существуют и более сложные случаи. Например, понятия «путь», «время» и «скорость» в онтологии механического движения связаны между собой соотношениями, которые выражаются только с помощью некоторой функции. Для такого рода задач необходимо уметь строить онтологии, которые выражают более сложные зависимости. И конечно, для этого необходим специальный язык, который включает эти возможности выражения. Обычно, в качестве такого языка используется та или иная разновидность языка исчисления предикатов, т.е. определяемая онтология представляет собой логическую теорию. В качестве примера рассмотрим язык Knowledge Interchange Format (KIF) [19], который используется в системе Ontolingua [14] в качестве языка определения онтологий. Впоследствии (глава 4), мы изучим этот язык более подробно, здесь только рассмотрим некоторые его идеи, понимание которых необходимо для усвоения примеров.

Язык KIF фактически представляет собой исчисление предикатов, содержащее функциональные символы и равенство. KIF базируется на языке LISP [18], поэтому имеет префиксную нотацию. KIF концептуализирует предметную область на основе трех основных понятий: «множество», «отношение» и «функция».

Множество представляет собой абстракцию предметной области, а именно: считается, что предметная область представляет собой множество всех *объектов* в мире, реально существующих или гипотетических. Ввиду того, что предметная область в языке KIF представляет собой абстракцию, разрешается чтобы различные пользователи имели свои предметные области, однако при этом все такие предметные области должны включать следующие базовые объекты:

- Слова. Иначе говоря, слова являются объектами любой предметной области, концептуальная схема которой моделируется языком KIF.
- Все комплексные числа.

- Все конечные списки объектов предметной области.
- Все множества объектов предметной области.
- Специальный объект, обозначаемый $\perp$, который считается значением любой функции, которая была вызвана с аргументами, для которых эта функция не определена.

Функция— это вид соотношения между объектами. С каждой конечной последовательностью объектов, которые называются *аргументами*, функция ассоциирует некоторый объект, называемый *значением* функции. Формально говоря, функция представляет собой множество конечных списков объектов, которые представляют собой возможные аргументы функции и последний элемент каждого списка— это значение функции для данного набора аргументов. Например, функция «2*» представляет собой множество двухэлементных списков вида $\langle 4, 8 \rangle$ или $\langle 17, 34 \rangle$, где первый элемент списка— это аргумент функции, а второй элемент— ее значение.

Отношение представляет собой другой вид связи между объектами предметной области. Отношение— это произвольное множество конечных списков объектов предметной области, причем эти списки в общем случае могут быть переменной длины. Про элементы каждого списка говорят, что они *удовлетворяют* данному отношению или, что отношение *выполняется* на этих элементах. Например, отношение «<» содержит списки $\langle 2, 3 \rangle$ и $\langle 45, 102 \rangle$, но не содержит списка $\langle 5, 2 \rangle$.

Язык KIF описывает онтологию предметной области используя отношения и функции. С помощью этих базовых понятий можно описать также и типы. Типы представляют собой специальные объекты предметной области. Для выделения типов среди объектов используется одноместное отношение «Class». Если данный объект удовлетворяет отношению «Class», то он и считается классом. Но при этом каждый класс сам по себе представляет одноместное отношение, которое выделяет множество его экземпляров. Отношения можно связывать друг с другом с помощью логических операций. Это обычные логические связки «или» (or), «и» (and) и отрицание (not), а также операция логического следования (импликация, т.е. если истинно A , то истинно также и B), обозначаемая в языке KIF оператором « $\Rightarrow$ » ($A \Rightarrow B$), а также его эквивалентами « $\Leftarrow$ » (следование в обратную сторону, т.е. $A \Leftarrow B$ это все равно, что $B \Rightarrow A$), « $\Leftrightarrow$ » (следование в обе стороны, т.е. $A \Leftrightarrow B$ эквивалентно одновременному выполнению $A \Rightarrow B$ и $B \Rightarrow A$). Также присутствует равенство. Переменные начинаются с символа «?». Язык KIF мы подробно опишем в главе 4, посвященной описанию системы

Ontolingua, здесь же удовлетворим любопытство читателя и приступим к обсуждению конкретных примеров.

Пример 1.1. (*Частичный порядок*)

Частичный порядок часто встречается на практике. Типичной является ситуация, что некоторые объекты сравнимы друг другом, а другие нет. При этом, каждый объект сравним с самим собой. Это свойство называется *рефлексивностью*. Другие два свойства отношения частичного порядка— *антисимметричность* и *транзитивность*, формально описаны чуть ниже. Обозначим отношение частичного порядка через « $\leq$ ». Математически, три его свойства можно выразить следующим образом:

1. *Рефлексивность*. Для любого объекта a выполняется $a \leq a$.
2. *Антисимметричность*. Если $a \leq b$ и $b \leq a$, то выполняется $a = b$.
3. *Транзитивность*. Если $a \leq b$ и $b \leq c$, то выполняется $a \leq c$.

Итак, частичный порядок— это любое двуместное отношение, для которого выполняются эти три свойства. Можно сначала задать «рефлексивное отношение», «антисимметричное отношение» и «транзитивное отношение», и затем определить отношение частичного порядка, как такое отношение, которое одновременно является рефлексивным, антисимметричным и транзитивным. Вот как это сделано в системе Ontolingua:

```
(defrelation partial-order
  (<=> (partial-order ?r ?s)
    (and (transitive ?r ?s)
      (asymmetric ?r ?s)
      (reflexive ?r ?s))))
```

В соответствии с синтаксисом языка Lisp, функциональные выражения выражения вида $f(x, y)$ представляются в виде списка `(f x y)`, в котором первый элемент— это имя функции, а остальные элементы списка представляют его аргументы. Элементы списка отделяются пробелами. Для определения отношения используется ключевое слово «defrelation». Определяется отношение частичного порядка `partial-order`, задаваемое переменной «?r» на множестве «?s». Отношение «?r» является на множестве «?s» частичным порядком тогда и только тогда, когда оно

является на множестве «?s» одновременно транзитивным, антисимметричным и рефлексивным. Конечно, еще необходимо определить три последних отношения. Приведем, например, определение для рефлексивного отношения:

```
(defrelation reflexive
  (<=> (reflexive ?r, ?s)
    (and (binrel ?r, ?s)
      (forall ?x)
        (=> (member ?x, ?s)
          (holds ?r, ?x, ?x))))))
```

Как и выше, отношение «?r», определенное на множестве «?s», является рефлексивным тогда и только тогда, когда выполняются одновременно два условия:

1. «?r» представляет собой бинарное отношение (binrel ?r, ?s), т.е. имеет всегда два аргумента.
2. Для каждого объекта «?x» (forall ?x) множества «?s» (member ?x, ?s) выполняется: «?x» сравним с самими собой (holds ?r, ?x, ?x).

Наверное, единственное что здесь требует комментария, это отношение «holds», которое используется для проверки того, входит ли список аргументов в отношение, заданное как первый параметр.

☒

***Изменяющиеся онтологии**

До сих пор мы игнорировали тот факт, что онтологии могут изменяться с течением времени. Примеры онтологий, которые были приведены выше, представляют собой как-бы снимки концептуальной схемы, которые не отражают ее изменений, происходящих во времени. Между тем, концептуальные схемы, как правило, изменяются, появляются новые понятия и исчезают некоторые старые, меняется терминология, то, что считалось когда-то важным и значительным, со временем перестает быть таковым, зато то, на что ранее никто не обращал внимания, вдруг обретает особый смысл и привлекает внимание людей. Хорошо было бы заложить эти возможности изменения концептуальной схемы в инструмент моделирования, с помощью которого производится построение ее онтологии, чтобы естественным образом эти изменения отображать. Конечно, существуют задачи, когда в самой онтологии задавать такие возможности нет нужды, проще просто выпустить новую версию онтологии, отражающую произошедшие изменения. Так, наверное, для рассмотренных выше классификаций, привнесение в онтологию возможностей ее изменения выглядит излишеством, только затрудняющим понимание основных сущностей. Но для многих

других задач, теснее соприкасающихся с реальностью, инструментарий поддержки изменений несомненно необходим.

Другой факт, который мы игнорировали до сих, состоит в том, что онтологии—это социальный продукт, т.е. результат коллективного труда. Онтология может создаваться одним человеком, но полезна она только тогда, когда используется другими людьми. Это означает, что существует договоренность между членами социума относительно содержания концептуальной схемы, которая строится в данный момент. Мы говорили о понятиях как о базовых элементах, на которых основано понимание. Но понятия становятся таковыми только тогда, когда оформляется договоренность между членами сообщества относительно их истолкования. До тех пор, понятия представляют собой только индивидуальные переживания (смыслы). Оформление смысла в понятие происходит посредством обсуждения, в процессе которого понятие уточняется и приобретает четкую форму. Результат такого обсуждения—это договоренность между членами сообщества о том, каким образом они соотносят свои индивидуальные переживания друг с другом. Было бы хорошо, чтобы программная система построения онтологий предоставляла возможности их коллективного редактирования. Реализация таких возможностей имеет свои трудности и подводные камни.

В процессе изменения типов может меняться и их объем, т.е. могут появляться новые экземпляры, другие экземпляры могут исчезать. С этой точки зрения, моделировать тип с помощью множества довольно неудобно, ведь множества определяются своими элементами, а последние могут быть еще не все известны⁷. Чтобы адекватно выразить свойство неполноты описания типа, необходим соответствующий формальный аппарат. Таким формальным аппаратом представляется *теория категорий* [7]. Теория категорий—это раздел математики, оперирующий математическими конструкциями под названием «категория». Категория содержит два вида элементов: объекты и морфизмы. Морфизмы выражают связи между объектами. Так, категория множеств содержит в качестве объектов множества, а в качестве морфизмов—отображения множеств друг в друга. Каждому морфизму соответствует два объекта данной категории: объект, ассоциированный с началом морфизма, и объект, ассоциированный с концом морфизма. Для категории множеств, где объектами выступают множества, а морфизмами—отображения множеств, объект, ассоциированный с началом морфизма—это область определения данного отображения-морфизма, а объект ассоциированный с концом морфизма, соответственно,—область значений. Традиционно, морфизмы обозначаются строчными буквами латинского алфавита, а объекты—прописными буквами латинского алфавита (нотация, унаследованная от традиционных обозначений множеств и функций между ними). Будем писать $f: A \rightarrow B$ для обозначения морфизма f с началом в объекте A и концом в объекте B . Между морфизмами должна быть задана операция *композиции* или умножения морфизмов, а именно: если $f: A \rightarrow B$ и $g: B \rightarrow C$, то существует морфизм $h = fg: A \rightarrow C$ —композиция морфизмов f и g , где конец первого морфизма совпадает с началом второго (композицию морфизмов f и g будем обозначать как fg или как $f \circ g$). Очевидно, что в нашем примере категории множеств композиция морфизмов совпадает с композицией функций. Морфизмы

⁷Поэтому в программах на объектно-ориентированных языках класс—это не множество: класс определяется именем, а не его элементами. У двух разных классов может быть в некоторый момент времени, случайно, одинаковое множество известных элементов (например, пустое).

между объектами должны удовлетворять определенным требованиям, а именно:

- Для каждого объекта A должен быть задан тождественный морфизм $id_A: A \rightarrow A$ такой, что для любых $f: B' \rightarrow A$ и $g: A \rightarrow B''$ выполняется $f \circ id_A = f$ и $id_A \circ g = g$. Иначе говоря, композиция тождественного морфизма с любым другим дает в результате все тот-же искомый морфизм. В точности так действует *тождественное отображение* на любом множестве (т.е. отображение, переводящее каждый элемент множества в него самого).
- Композиция морфизмов обладает свойством ассоциативности: если заданы морфизмы $f: A \rightarrow B$, $g: B \rightarrow C$ и $h: C \rightarrow D$, то должно выполняться $f(gh) = (fg)h$. Иначе говоря, композицию трех (а значит, и любого числа) морфизмов можно брать в любом порядке, в результате получим одну и ту же стрелку.

Оказывается, любую теоретико-множественную конструкцию можно определить с использованием инструментария теории категорий, т.е. через набор объектов, которыми оперирует данная конструкция, и набор морфизмов между этими объектами. Такие представления бывают весьма неожиданными и зачастую полезны для прояснения данной математической конструкции. Понятие категории лежит в основе любой теоретико-множественной конструкции, что и определило название данного математического формализма,— категория служит основанием математических построений точно также, как философские категории Канта и Аристотеля для онтологических теорий этих философов.

В теории категорий можно определить аналоги различных операций над множествами, такими как декартово произведение множеств или их пересечение. Декартово произведение $X \times Y$ —это также множество, в данном случае, множество пар, а значит, объект категории множеств. Для любых двух множеств существует их декартово произведение, поэтому декартово произведение можно представить как специальную операцию, сопоставляющую любым двум объектам категории третий объект. Такие операции носят название *универсальные конструкции* или пределы. Для множеств можно определить их декартово произведение, сумму, возведение в степень и другие операции. Категория, в которой для объектов определены все те операции, которые определены для обычных множеств, называются *топосом*. Таким образом, топос представляет собой категорное представление понятия «множество всех множеств» или, как говорят математики,— «универсума». Объекты топоса определяются не своим содержанием (как в случае множеств), а взаимоотношением с другими объектами. Топос хорошо подходит для моделирования онтологий, в которых понятия задаются с помощью неполных типов. Понятия можно представлять как объекты некоторого топоса, а различные соотношения между понятиями— в виде морфизмов. Новые понятия могут быть образованы с помощью универсальных конструкций. Например, пусть в топосе есть два специальных объекта A и B , первый моделирует понятие «мужчина», а второй— понятие «женщина». Тогда можно сконструировать объект, обозначающий понятие «возможные семейные пары», который находится с объектом $A \times B$ в специальном отношении, называемом «отношением подобъекта». Отношение подобъекта— это категорный аналог того факта, что одно множество является подмножеством другого. Чтобы ввести понятие «реальные семейные пары» необходимо построить для этого специальный объект, который из возможных пар выбирает только те, которые присутствуют в действительности. Для этого надо использовать *элементы объекта*— морфизмы из специального *конечного объекта*, аналога одноэлементного множества.

Иногда бывает, что даже структура топоса не вполне отражает процессы конструирования новых понятий. Рассмотрим, например, конструкцию образования новых понятий методом *понятийного смешивания* [26]. Понятийное смешивание часто бывает несимметрично. Например, если имеется два понятия: «лодка» и «дом», то понятие «дом-лодка» это совсем не тоже самое, что понятие «лодка-дом». С помощью конструкции предела такое несимметричное смешивание выразить трудно, т.к. пределы симметричны относительно объектов и морфизмов, на основе которых они конструируются. Поэтому необходимо ввести в определениях понятий возможность выделить главные (обязательные) свойства и модифицировать операции смешивания понятий с учетом этого выделения. Соответственно, необходимо модифицировать определения математических конструкций, моделирующих операцию смешивания понятий. Такие попытки уже были проведены Джозефом Гогеном, в результате появились так называемые *2/3-категории* [25; 26].

Онтологическое моделирование на основе формализма теории категорий представляет собой отдельную большую тему. В данной книге эти вопросы подробно затрагиваться не будут.

1.4 Что изложено в книге

Книга посвящена изложению современных систем построения онтологий в различных областях инженерного знания. Современные подходы к онтологическому моделированию основаны, главным образом, на представлении концептуальной схемы на языке исчисления предикатов первого порядка, в который добавлены некоторые свойства языков второго порядка: различные ограничения на свойства отношений (предикатов)⁸. Таким образом, описания наиболее популярных систем управления онтологиями, данные в книге, тем или иным образом касаются исчисления предикатов. Различные аспекты, связанные с языками исчисления предикатов, рассматриваются по мере необходимости их упоминания в изложении описываемых систем управления онтологиями.

⁸Можно ввести ограничения на количество элементов отношения. Например, каждый человек имеет отца и мать, причем не более, чем одного отца и одной матери. Если, для указания того, что данная персона имеет в качестве отца конкретного мужчину, используется двуместное отношение *ИмеетОтца* (берущее объект персоны как первый параметр, а объект отца—как второй), то необходимо также определить и ограничение, выражающее то факт, что одна и та же персона имеет в точности одного отца. Это можно сделать, указав, что данное отношение должно вести себя как функция, сопоставляющая первому параметру (персоне) второй параметр (отца). Функция, как известно, не может давать в соответствие каждому своему аргументу более одного значения, поэтому указанное выше ограничения будет выполняться. В языке OWL отношения с такими ограничениями так и называются—функциональные.

Первая глава представляет собой расширенное введение в предмет изложения. В разделе 1.1 излагается мотивация изучения понятия «онтология» и приводятся конкретные примеры задач, когда без использования онтологий решение этих задач затруднительно. Во разделе 1.2 дается определение онтологии, а также излагаются различные понятия, связанные с этим определением. Раздел 1.3 посвящен изложению различных типов онтологий. В данном разделе приводятся примеры классификаций и таксономий, а также более сложных типов онтологий, таких как онтологии с соотношениями и изменяющиеся онтологии. И наконец, раздел 1.4 посвящен описанию того, что будет изложено в последующих главах книги.

Вторая глава представляет собой сборник из трех несвязанных между собой разделов. В разделе 2.1 излагаются философские основы понятий, связанных с понятием концептуализации. Здесь излагается в кратком виде теория прототипа, а также даются основы философского направления под названием *концептуальный релятивизм*. На основании изложенного, дается определение концептуальной схемы, как результата концептуализации инженерной задачи. Раздел 2.2 представляет собой обзор известных реализаций программных систем управления онтологиями. Каждая из таких систем содержит в качестве описания общих понятий свою онтологию верхнего уровня. Эти онтологии действительно представляют собой онтологии, а не концептуальные модели в терминологии Гуарино [34]. В разделе также дается описание «бриллианта Совы» — онтологии верхнего уровня, разработанной известным американским исследователем Джоном Совой. Онтология дана в виде диаграммы понятий, внешне напоминающим ограненный бриллиант, отсюда и название. И, наконец, раздел 2.3 представляет собой собрание различных методологий построения онтологий. Рассматриваются три методологии:

- Методология, предложенная разработчиками языка UML [24], созданного специально для построения моделей программных систем.
- Методология, предложенная Грубером в статье «Предполагаемые принципы проектирования онтологий, используемых для обмена знаниями» [32]. Эта методология опирается на несколько основополагающих принципов построения онтологий. Изложению этих принципов и посвящен данный подраздел.
- Методология, основанная на так называемом BORO-методе разработки онтологий. BORO (Business Object Reference Ontology)

методология [15] была предложена для решения задачи реорганизации существующих информационных систем в информационные модели. В разделе дается описание этой методики и ее основных принципов.

Глава 3 посвящена изложению языка OWL [10] и связанных с этим языком концепций. Собственно, языку OWL уделено не так много внимания. Основное предназначение данной главы состоит не в том, чтобы научить конструировать онтологии на языке OWL, а в том, чтобы познакомить читателя с основными понятиями, которые содержит в себе этот язык. В разделе 3.1 даются базовые принципы Semantic Web (Умного Веб), для реализации которого и предназначен язык OWL. Язык OWL основан на синтаксисе языка XML [11], поэтому в разделе 3.2 дается краткое описание идей этого языка. В разделе 3.3 излагаются основы языка RDF [9], который является ядром для языка OWL. В данном разделе также описывается язык SPARQL [20] запросов к RDF хранилищам. Раздел 3.4 посвящен изложению понятий, связанных непосредственно с языком OWL. В разделе дается краткое изложение основ языка, а также различных диалектов языка OWL. В разделе 3.5 описан интерфейс и предложения редактора Protege. Protege представляет собой редактор онтологий, записанных на языке OWL. В разделе также дается пример онтологии, записанной на языке OWL, иллюстрирующий изложенные в главе идеи. В последнем разделе главы даются ссылки на некоторые «свободные» реализации RDF хранилищ с открытыми исходными кодами, которые заинтересованный читатель может использовать в разработке своих систем.

В главе 4 излагаются различные идеи, связанные с системой управления онтологиями Ontolingua [31]. Раздел 4.1 посвящен изложению идей, послуживших побудительным мотивом для создания системы. Раздел 4.2 содержит описание языка KIF (Knowledge Interchange Format) [19]. Описание основано на драфте стандарта dpANS [3] языка KIF. Таким образом, данный раздел можно использовать как неофициальную русскоязычную версию стандарта языка KIF. Раздел 4.3 содержит описание той версии языка KIF, которая применяется в системе Ontolingua. Этот диалект имеет несколько отличий от оригинального KIF, эти различия описаны в данном разделе. Раздел 4.4 содержит пример онтологии, записанной на языке Ontolingua KIF, приведенный для иллюстрации изложенных в главе идей. В разделе 4.5 дается описание Онтологического Фреймового Редактора—инструмента редактирования онтологий, предоставляемого системой Ontolingua. Фреймовый Редактор реализован как Web сервер, предоставляющий свой интер-

фейс в виде Web-страниц с формами. Описание функциональности этого редактора составляет содержание данного раздела.

Глава 5 посвящена изложению идей, реализованных в системе управления знаниями Сус [21]. Система Сус представляет собой базу знаний, основанную на языке исчисления предикатов первого порядка. Описанию основных свойств системы и истории ее развития посвящен раздел 5.1. Для предоставления интерфейса к базе знаний, добавления новых знаний и их систематизации, в Сус реализован специальный язык, названный СусL [22]. Этот язык описан в разделе 5.2. В разделе 5.3 описана система логического вывода, которая используется в базе знаний Сус. В системе используется метод резолюций в качестве базового метода построения логического вывода, но также реализовано множество эвристических методов, позволяющих уменьшить перебор вариантов во время выполнения алгоритма логического вывода. Раздел 5.4 содержит описание онтологии верхнего уровня базы знаний системы Сус. Как и всякая развитая система управления знаниями, система Сус должна каким-то образом их систематизировать. Для этого в онтологии Сус описана сложная система общих понятий, а также системы понятий различных предметных областей, и конкретные знания добавляются в рамках определенной предметной области. В разделе 5.5 описан проект OpenСус [16], представляющий собой открытую часть базы знаний Сус. База знаний Сус вообще предоставляется за деньги, но некоторая ее часть, включающая в себя онтологию верхнего уровня, доступна для публичного доступа под именем OpenСус.

1.5 Список литературы

1. Бобров Е. Г. Карл Линней. 1707—1778. — Л.: Наука. 1970. — 285 с.
2. Гоклениус Р. *Lexicon philosophicum, quo tanquam clave philisophiae fores aperiunter*. Fransofurti, 1613.
3. Драфт стандарта dpANS языка KIF.
<http://logic.stanford.edu/kif/dpans.html>.
4. . Клауберг И. *Metaphysika de ente, quae rectus Ontosophia*. Амстердам, 1991.
5. Когаловский М., Калиниченко Л. Концептуальное моделирование и онтологические модели. В сборнике *Онтологическое моделирования, труды симпозиума в г. Звенигород, 19-20 мая, 2008*.
6. Линней К. Система природы. Царство животных, ч. 1–2. СПб, 1804–1805.
7. Маклейн С. Категории для работающего математика. Перевод под редакцией В. А. Артамонова. М., Физматлит, 2004.
8. Менделеев Д. И., Периодический закон. Основные статьи, М., 1958.
9. Описание концепций языка RDF на сайте W3.
<http://www.w3.org/TR/rdf-concepts/>
10. Описание языка OWL на сайте W3. <http://www.w3.org/TR/owl-ref/>.
11. Описание языка XML на сайте W3. <http://www.w3.org/XML/>.
12. Сайт Каталога Яндекса. <http://catalog.yandex.ru/>.
13. Сайт компании Cycorp. <http://www.cyc.com>.
14. Сайт Онтолингва. <http://www.ksl.stanford.edu/software/ontolingua/>.
15. Сайт, посвященный BORO методологии.
<http://www.boroprogram.org>.
16. Сайт проекта OpenCyc. <http://opencyc.org>.
17. Сайт проекта WordNet. <http://wordnet.princeton.edu>.

18. Сайт языка LISP. <http://www.lispworks.com/documentation/common-lisp.html>.
19. Сайт KIF. <http://ksl.stanford.edu/knowledge-sharing/kif/>.
20. Страница на сайте консорциума W3, посвященная языку SPARQL. <http://www.w3.org/2001/sw/DataAccess/>.
21. Страница описания онтологии Сус. http://www.cyc.com/cyc/technology/whatis_cyc_dir/whatdoes_cyc_know.
22. Страница описания языка СусL на сайте проекта Сус. <http://www.cyc.com/cycdoc/ref/cycl-syntax.html>.
23. Труды симпозиума «Онтологическое моделирование». М.: ИПИ РАН, г. Звенигород, 19-20 мая, 2008.
24. Буч Г., Рамбо Д., Джекобсон А. Язык UML. Руководство пользователя: Пер. с англ.— М.: ДМК, 2000.
25. Goguen J. What Is a Concept?. ICCS, pp. 52-77, 2005.
26. Goguen J, Harrell F. Foundations for active multimedia narrative: Semiotic spaces and structural blending. Interaction Studies: Social Behaviour and Communication in Biological and Artificial systems, 2005.
27. Clancey W. The Knowledge Level Reinterpreted: Modelling Socio-Technical Systems. International Journal of Intelligent Systems, 8: 33-496 1993.
28. Codd E. A Relational Model of Data for Large Shared Data Banks. Commun. ACM 13(6): pp. 377-387 (1970)
29. Fonteseca F., Martin J. Learning the differences between Ontologies and Conceptual Schemas through Ontology-Driven Information Systems. Journal of the Association for Information Systems—Special Issue on Ontologies in the Context of IS. Volume 8, Issue 2, Article 3, February 2007.
30. Gruber T. R. The role of common ontology in achieving sharable, reusable knowledge bases. In J. A. Allen, R. Fikes, and E. Sandewell, editors, Principles of Knowledge Representation and Reasoning — Proceedings of the Second International Conference, pp. 601-602. Morgan Kaufmann (1991)

31. Gruber T. R. Ontolingua: A mechanism to support portable ontologies.
// Stanford University, Knowledge Systems Laboratory, Technical
Report KSL-91-66, Mar 1992.
32. Gruber T. R. Toward Principles for the Design of Ontologies Used for
Knowledge Sharing. *International Journal Human-Computer Studies*,
43(5-6): pp. 907-928. (1995)
33. Guarino N. Formal Ontology, Conceptual Analysis and Knowledge
Representation. *International Journal of Human-Computer Studies*,
43(5-6): pp. 625-640. (1995).
34. Guarino N., Formal Ontology in Information Systems. *Proceedings of
FOIS'98, Trento, Italy, 6-8 June 1998*. Amsterdam, IOS Press, pp. 3-15.
35. Liskov B. *Abstraction and specification in program development*, MIT
Press, Cambridge, MA, 1986.
36. Newell, A. The Knowledge Level. *Artificial Intelligence*, 18: 87-127
1982.
37. Smith B. Ontology. In Floridi L. (ed.) *The Blackwell Guide to the
Philosophy of Computing and Information*. pp. 155-167. 2003.